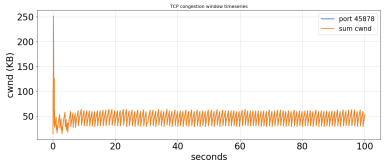
CSC 458 - Fall 2025 - Programming Assignment 2: Bufferfloat

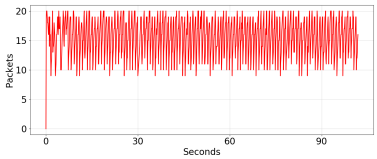
Chloe Lam, 1007650061

November 24, 2025

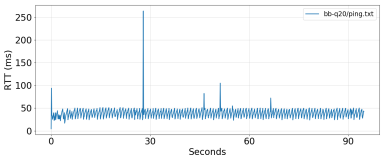
Original Experiment Outputs:



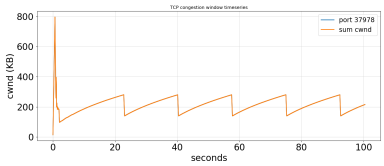
CWND (q=20)



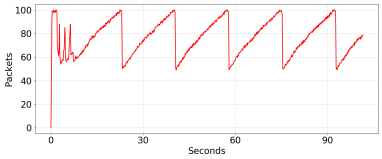
Queue (q=20)



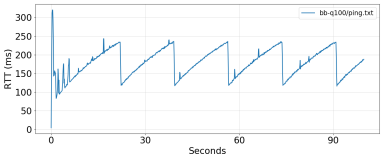
RTT (q=20)



CWND (q=100)



Queue (q=100)



RTT (q=100)

Figure 1: Comparison of CWND, queue size, and RTT for buffer sizes 20 and 100.

4

1. **[2 points]** Why do you see a difference in webpage fetch times with small and large router buffers?

Queue Size	Mean (s)	Std Dev (s)
20	0.568216	0.077315
100	1.135607	0.589932

Table 1: Webpage fetch time statistics for small (q=20) and large (q=100) buffers.

There are much longer webpage fetch times when the router buffer is large ($q = 100$) compared to smaller buffer sizes ($q = 20$), about twice as slow on average at 1.14s vs 0.57s. This is because a larger buffer means packets stay in the queue longer, increasing RTT (up to 200ms when $q = 100$). A higher RTT causes slower ACKs, and because TCP depends on RTT to adjust its congestion window, this means slower webpage transfers. Meanwhile, a small buffer ($q = 20$) prevents the longstanding queues from forming, meaning RTT will stay lower and more stable, allowing TCP to recover quickly and complete webpage fetches faster.

2. **[2 points]** Bufferbloat can occur in other places such as your network interface card (NIC). Use `ifconfig` or `ip link show` on your Mininet VM to identify the transmit queue length (txqueuelen) of an interface such as enp0s1 and report the output. For this queue size and a draining rate of 100 Mbps, what is the maximum time a packet might wait in the queue before it leaves the NIC?

```
mininet@mininet-vm:~/csc458-pa-fall-2025$ ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp0s1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 1e:da:ae:10:e8:05 brd ff:ff:ff:ff:ff:ff
```

The transmit queue length of interface `enp0s1` is: $Q = 1000$ packets.

We assume:

- Transmit Queue Length $Q = 1000$ packets
- MTU-sized packets of approximately $L = 1500$ bytes
- Draining Rate of $C = 100$Mbps $= 100 \times 10^6$ bit/s $= \frac{100 \times 10^6}{8}$ bytes/s $= 12,500,000$ bytes/s

The maximum time a packet might wait in the NIC queue is:

$$\text{time} = \text{data}/\text{rate}$$
$$T_{max} \approx \frac{Q \cdot L}{C}$$
$$T_{max} = \frac{1000 \cdot 1500}{12500000}$$
$$T_{max} = 0.12s$$

Thus, at a 100 Mbps drain rate, a packet may wait up to $T_{max} \approx 0.12s = 120ms$.

3. **[3 points]** Analyze your plots of CWND, RTT, and queue size.

   - Derive or express a symbolic equation showing how RTT varies with queue size (ignore computation overheads in ping that might affect the final result).

     Let RTT_0 denote the base round-trip time (propagation + transmission delay with an empty queue). If the bottleneck queue contains q packets, each of size L bytes, and the bottleneck drains at capacity C (bytes/s), then the queueing delay is $D_{\text{queue}}(q) = \frac{q \cdot L}{C}$.

     Thus, the total RTT is $\mathrm{RTT}(q) = \mathrm{RTT}_0 + D_{\text{queue}}(q) = \mathrm{RTT}_0 + \frac{q \cdot L}{C}$.

     This shows that RTT increases linearly with queue occupancy, which is consistent with the shapes of the RTT curves in both $q = 20$ and $q = 100$.

   - Explain how CWND oscillations correspond to RTT spikes in your plots.

     As CWND increases, packets are injected into the network faster than the bottleneck link can drain them, causing the queue length at the bottleneck to rise which leads directly to an increase in RTT.

     This relationship is clearly visible in the plots:

     - When CWND increases, the queue gradually fills, producing a corresponding upward ramp in RTT.
     - When CWND drops sharply (after a packet loss and multiplicative decrease), the queue drains quickly and RTT returns to near its base value RTT_0.
     - Each CWND sawtooth cycle therefore produces a matching "triangle" in the queue plot and a corresponding spike or ramp in the RTT plot.

   - Discuss how buffer size influences both TCP performance and webpage fetch times.

     Small buffer (q=20): The queue rarely grows large, so RTT remains relatively low and oscillates in a narrow range. CWND cycles are short and losses occur frequently, which limits throughput but avoids persistent queueing delay. The average webpage fetch time is approximately 0.57 s.

     Large buffer (q=100): The large queue allows many more packets to accumulate before a loss occurs. This produces long-standing queues and much larger RTT spikes (over 200 ms). CWND grows significantly larger before collapsing. Although this reduces the rate of packet loss, it introduces substantial queueing delay. The average webpage fetch time increased to approximately 1.14 s, more than twice as large than the small-buffer configuration.

4. **[3 points]** Identify and describe two ways to mitigate bufferbloat. For each, explain the trade-offs and propose an experiment using your Mininet setup to evaluate its quantitative effectiveness. Clearly document your experimental design, including parameter choices, measurement methodology, and justification for your conclusions, so that the TA can reproduce your results.

Method 1: Using Smaller Buffers (BDP-Based Buffer Sizing)

A simple way to mitigate bufferbloat is to reduce the router's buffer size so that it is closer to the bandwidth–delay product (BDP) rather than significantly larger than it. If the link capacity is $C = 10$ Mbps and the base RTT is roughly $\mathrm{RTT}_0 = 4$ ms, then the BDP is:

$$\mathrm{BDP} = C \cdot \mathrm{RTT}_0 = \frac{10 \times 10^6}{8} \cdot \frac{4}{1000} = 5000 \text{ bytes}.$$

So, # of packets $= \frac{\text{BDP}}{\text{Packet Size}} = \frac{5000 \text{ bytes}}{1500 \text{ bytes}} \approx 3.3$ packets. The link only needs about 3 to 4 packets to stay fully utilized. This is much smaller than our two experiments of $q = 20$ and $q = 100$ packets. Reducing the buffer size closer to $q = 3/4$ prevents the queue from building up excessively, keeping queueing delay and RTT much lower.

Pros	**Cons**
• Smaller queues reduce RTT • Improve latency-sensitive traffic (e.g., webpage fetches) • Limit bufferbloat	• More packet drops during bursty traffic • If the buffer is too small to absorb natural traffic variability – May reduce throughput slightly – May slightly reduce throughput

Table 2: Trade-offs of reducing router buffer size.

In `scripts/run.sh`, we test 2 queue sizes q=10 and q=60 (i.e. `QSIZES = (10 60)`). All other parameters (bottleneck bandwidth 10 Mbps, delay 1 ms per link, TCP Reno, experiment duration 100 s) are held constant.

We evaluate its quantitative effectiveness by analyzing statistics like:

- Minimum/Average/Maximum RTT
- Packet loss
- Webpage fetch times (Individual/Mean/Standard Deviation)

After running the experiment, we get this result:

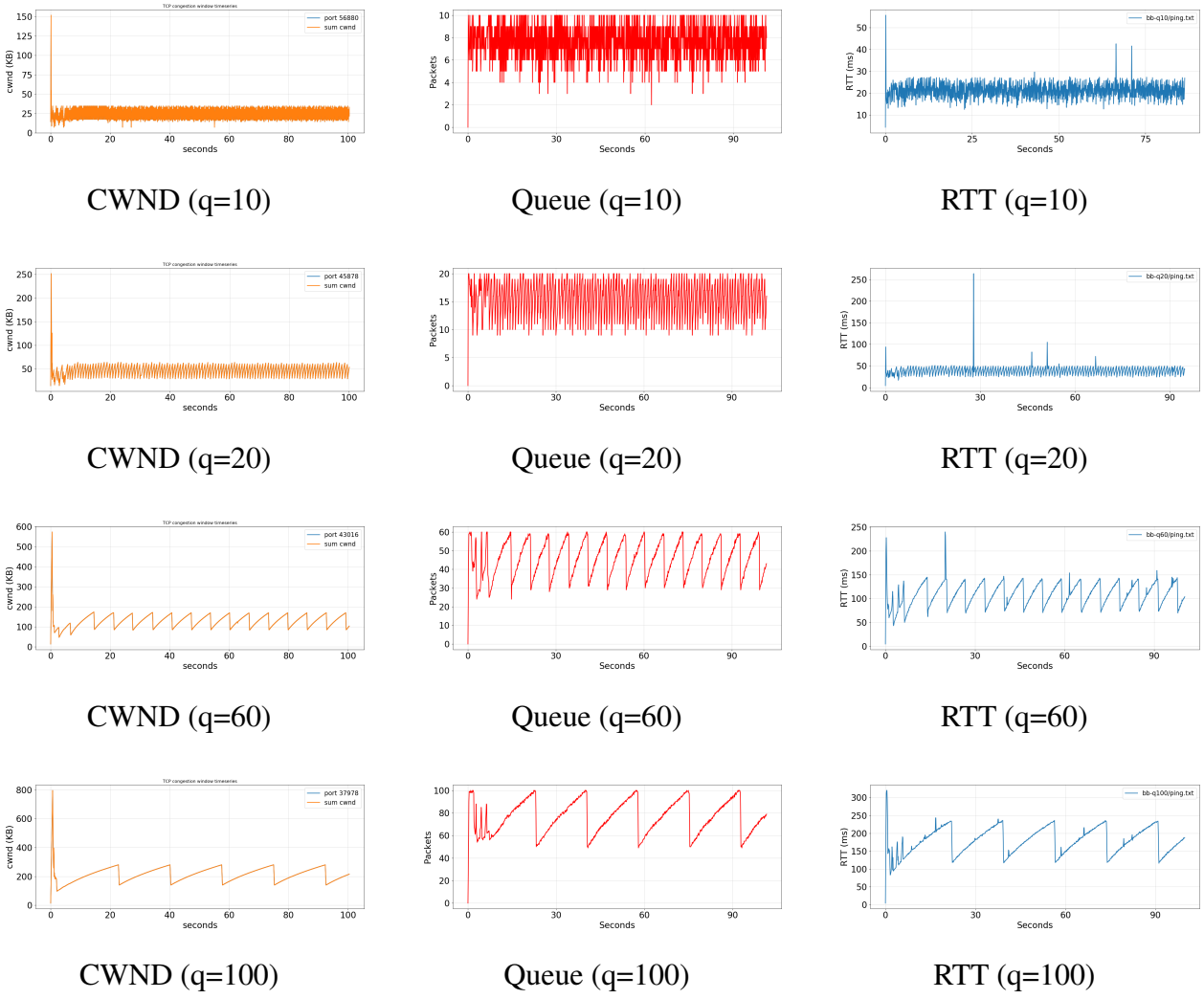


Figure 2: Comparison of CWND, queue size, and RTT for buffer sizes 10, 20, 60 and 100.

Queue Size	Mean (s)	Std Dev (s)
10	0.696649	0.122844
20	0.568216	0.077315
60	1.240856	0.719720
100	1.135607	0.589932

Table 3: Webpage fetch time statistics for q=10, 20, 60 and 100 buffers.

We see that larger buffers overall yield larger standing queues and higher RTT, leading to slower webpage fetch times, which matches our results (mean fetch times 0.70s, 0.57s, 1.24s, and 1.14s for corresponding $q = 10, 20, 60, 100$).

Method 2: Active Queue Management (AQM) - ex. Controlled Delay (CoDel), Random Early Detection (RED)

Another way to mitigate bufferbloat is to use active queue management, which drops/marks packets before the buffer becomes full. This prevents persistent queues from forming even when the nominal buffer size is large.

Pros	Cons
• Significantly reduces queueing delay and RTT • Improves latency for short flows • Helps prevent large bufferbloat spikes	• Introduces more packet drops/marks than simple tail-drop • May slightly reduce throughput • Requires configuration and tuning

Table 4: Trade-offs of Active Queue Management (AQM).

In `bufferbloat.py`, we add some code after `net.pingAll():`

```
cmd = f"tc qdisc replace dev s0-eth2 root fq_codel limit {args.maxq}"
print("Installing fq_codel on s0-eth2 with:", cmd)
subprocess.run(cmd, shell=True, check=True)
```

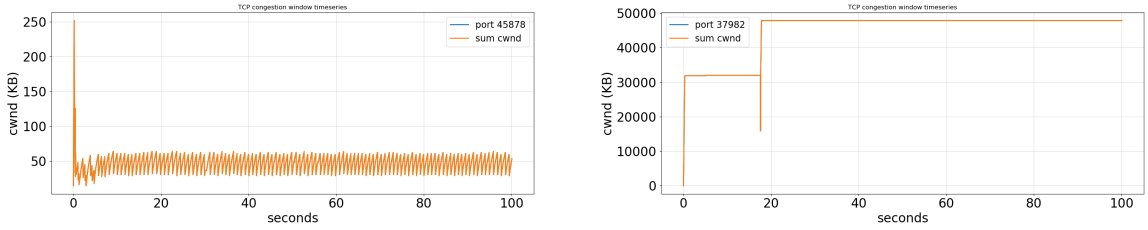
To avoid mix-up with non-AQM experiments, in `scripts/run.sh`, we set `dir="bb-aqm-q$qsize"`. All other parameters (bottleneck bandwidth 10 Mbps, delay 1 ms per link, TCP Reno, experiment duration 100 s) are held constant.

We evaluate its quantitative effectiveness by analyzing statistics like:

- Minimum/Average/Maximum RTT
- Packet loss
- Webpage fetch times (Individual/Mean/Standard Deviation)

After running the experiment, we get this result:

$q = 20$:



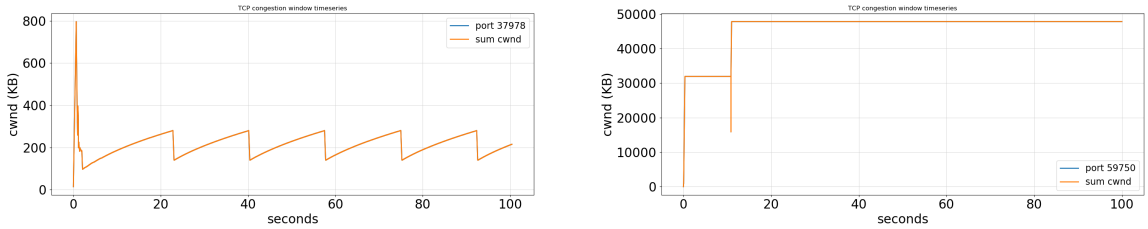
CWND (q=20) - without AQM CWND (q=20) - with AQM

Figure 3: Comparison of CWND with and without AQM.

	With AQM	**Without AQM**
Mean	0.721861	0.592751
Std	0.006779	0.117084

Table 5: Comparison of mean/std webpage fetch time statistics with and without AQM (CWND q=20).

$q = 100$:



CWND (q=100) - without AQM CWND (q=100) - with AQM

Figure 4: Comparison of CWND with and without AQM.

	With AQM	**Without AQM**
Mean	0.708620	1.135607
Std	0.001084	0.589932

Table 6: Comparison of mean/std webpage fetch time statistics with and without AQM (CWND q=100).

From these graphs we see that CWND hits a high value and stays almost perfectly flat. There are no sawtooth oscillations or repeated drop meaning that AQM actively controls the queue by early-dropping packets. It prevents the queue from building up, TCP never gets huge delays. Congestion remains stable so CWND stays stable.

5

Assumptions and Reference Formulas

- TCP congestion control follows AIMD.
- Bottleneck buffer is FIFO.
- CWND is measured in packets (MSS-sized payload). MSS explicitly used when converting to bytes/bits.
- Steady-state CWND cycles between $W_{min} = W_{max}/2$ and W_{max}.
- Steady-state packet loss probability: p (loss per packet sent). Note that loss occurs only due to buffer overflow.
- Round-trip time: $RTT = T_{prop} + T_{queue}$, with T_{prop} base propagation + processing delay, T_{queue} one-way queueing delay.
- Bottleneck capacity: C (packets/sec). Router buffer size: B (packets). Queue occupancy: Q, $0 \leq Q \leq B$.
- Packet serialization time at bottleneck: $\tau = 1/C$ (seconds/packet). Use C in packets/sec; convert to bits/sec via MSS if needed.

1. **[3 points] Sent packets per cycle** Consider the CWND-RTT graph. Determine what is the cycle between the minimum congestion window and the maximum congestion window. Assume that no random loss occurs. Now, by summing the packets sent over RTT, show that the total number of packets sent through a cycle is roughly equal to $N_{packets} \approx 3{W_{max}}^2/8$.

   Assuming AIMD sawtooth where the congestion window grows from $W_{\min} = W_{\max}/2$ to $W_{\max}$.

   The number of RTTs in one cycle is: # of RTTs $= W_{\max} - W_{\min} = W_{\max} - \frac{W_{\max}}{2} = \frac{W_{\max}}{2}$.

   The average congestion window over the cycle is: $\bar{W} = \frac{W_{\max}+W_{\min}}{2} = \frac{W_{\max}+W_{\max}/2}{2} = \frac{3W_{\max}}{4}$.

   The total number of packets sent in one cycle is approximately

$$N_{\text{packets}} \approx \bar{W} \cdot \#\text{ of RTTs} = \left(\frac{3W_{\max}}{4}\right)\left(\frac{W_{\max}}{2}\right) = \frac{3W_{\max}^2}{8}.$$

2. **[3 points] CWND and Throughput Modeling** Using the term derived above, show that the steady-state relationship between average CWND $\overline{W}$, loss probability p, and throughput is: $T \approx \frac{K \cdot MSS}{RTT\sqrt{p}}$ and compute constant K. State all assumptions.

   In steady state, we assume that each sawtooth cycle ends with one loss event. Then, the expected number of packets between losses is $1/p$. Equating this to the number of packets per cycle derived above, $\frac{1}{p} \approx N_{\text{packets}} \approx \frac{3W_{\max}^2}{8}$, we obtain $W_{\max} \approx \sqrt{\frac{8}{3p}}$.

   Then, the average congestion window over a cycle is $\bar{W} = \frac{3W_{\max}}{4} \approx \frac{3}{4}\sqrt{\frac{8}{3p}}$.

   TCP throughput means data per unit time, which is approximated by

$$T \approx \frac{\text{bytes per RTT}}{\text{seconds per RTT}} = \frac{\overline{W} \cdot MSS}{RTT} = \frac{3}{4}\sqrt{\frac{8}{3p}} \cdot \frac{MSS}{RTT}.$$

   Setting constant $K = \frac{3}{4}\sqrt{\frac{8}{3}} \approx 1.22$, we get:

$$T \approx \frac{3}{4}\sqrt{\frac{8}{3}} \cdot \frac{1}{\sqrt{p}} \cdot \frac{MSS}{RTT} = \frac{K \cdot MSS}{RTT\sqrt{p}}.$$

3. **[2 points] Queueing Delay and RTT** Considering the serialization delay, derive maximum one-way queueing delay for buffer B and link capacity C. Express measured RTT as a function of the minimum RTT of the network RTT_0, queue occupancy Q, MSS, and C. Specify any additional queueing assumptions if you make any.

   The serialization time per packet is $\tau = \frac{\text{packet size}}{\text{link rate}} = \frac{MSS}{C}$ seconds/packet.

   The one-way queuing delay is approximately $T_{\text{queue}}(Q) \approx Q \cdot \tau = Q \cdot \frac{MSS}{C}$ when the queue occupancy is Q packets. At maximum, with a full buffer of size B packets, the maximum one-way queuing delay is $T_{\text{queue}(B)} \approx B \cdot \frac{MSS}{C}$.

   Let RTT_0 denote the minimum RTT of the network (propagation + processing with negligible queuing). The measured RTT as a function of queue occupancy is then

   $$RTT(Q) \approx RTT_0 + T_{\text{queue}}(Q) = RTT_0 + \frac{Q \cdot MSS}{C}$$

   We assume a single FIFO bottleneck queue, a work-conserving link with constant rate C, and negligible queuing elsewhere.

4. **[2 points] Effect of Buffer Size on Web Fetch Times**

   Using the previous results, explain algebraically why increasing buffer B can increase short-transfer latency even if long-term throughput remains high.

   A short web transfer of size S bits has a flow completion time of approximately $RTT(Q) + \frac{S}{T}$, where $RTT(Q)$ is the round-trip time when the queue contains Q packets, and T is the throughput.

   We know the round-trip time grows with queue occupancy: $RTT(Q) \approx RTT_0 + \frac{Q \cdot MSS}{C}$

   If the bottleneck buffer is large and often full, then $Q \approx B$, which gives $RTT \approx RTT_0 + \frac{B \cdot MSS}{C}$

   Thus, increasing the buffer size B increases the queuing delay term $\frac{B \cdot MSS}{C}$ and therefore increases the RTT experienced by all flows.

   For short transfers, $\frac{S}{T}$ is small, so the flow completion time is dominated by the RTT at approximately $RTT_0 + \frac{B \cdot MSS}{C}$

   This shows that larger buffers can keep throughput T high while significantly increasing the latency of short web fetches.